Object-Oriented Principles

Abstraction, Polymorphism, Inheritance, Encapsulation, Composition (APIEC)

Object-oriented programming organizes code around objects—bundles of data and the operations that work on that data. These five principles form the foundation of OO thinking.

Abstraction

Abstraction means exposing only what matters and hiding the rest. A `Car` object might expose `start()`, `accelerate()`, and `brake()` without revealing anything about fuel injection or spark timing. Users interact with a simplified model, not the messy internals.

Good abstractions make code easier to use and understand. They let you think at the right level of detail for the problem at hand.

Polymorphism

Polymorphism means "many forms." Different objects can respond to the same message in different ways. If both `Dog` and `Cat` implement `speak()`, you can call `speak()` on any animal without knowing which type it is—each responds appropriately.

This enables writing code that works with abstractions rather than specific types. You can add new kinds of animals without changing the code that uses them.

Inheritance

Inheritance lets one class derive from another, acquiring its data and behavior. A `Manager` class might inherit from `Employee`, gaining all employee attributes while adding manager-specific ones.

Inheritance creates "is-a" relationships: a manager *is an* employee. It's powerful for code reuse but can create tight coupling between classes. Use it thoughtfully.

Encapsulation

Encapsulation bundles data with the code that operates on it and controls access to both. Instead of exposing raw data for others to manipulate, an object protects its internal state and provides methods to interact with it safely.

This prevents code from becoming tangled with assumptions about how other code stores its data. When internals change, encapsulated code limits the blast radius.

Composition

Composition builds complex objects by combining simpler ones. Rather than inheriting from a class, you hold a reference to an instance of it. A `Car` might *contain* an `Engine` rather than *being* a kind of engine.

Composition creates "has-a" relationships and tends to be more flexible than inheritance. It's easier to swap out components and avoids the fragile base class problem. The principle "favor composition over inheritance" reflects hard-won experience.

References

- Grady Booch et al., *Object-Oriented Analysis and Design with Applications* (2007) — Classic text on OO fundamentals.
- Joshua Bloch, *Effective Java* (2018) — Practical OO design guidance, especially on inheritance vs. composition.